

ADVANCED OPERATING SYSTEMS

Lab 2 — System-wide tracing

Vittorio Zaccaria

Politecnico di Milano

Syscall tracing

One workload, three observations

In Lab 1, GDB stopped the kernel. Here we record what happens while it runs.

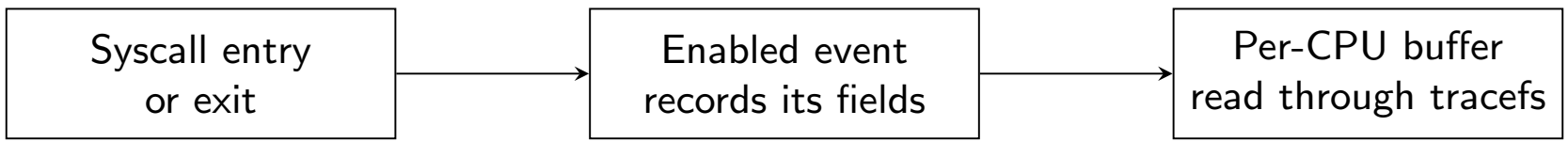
```
cat /init > /dev/null
```

1. **Syscalls:** which requests does the process make, and what do they return?
2. **Function graph:** which kernel functions execute inside one request?
3. **Perf:** how do we save the events and interpret their aggregation?

Commands run inside a fresh **amd64 QEMU guest**, unless marked otherwise. The captures use Linux and perf `5.16.0-rc1`, one virtual CPU, and a 584-byte `/init`. PIDs, addresses and timings vary.

Tracepoints, events and buffers

A **tracepoint** is a predefined instrumentation site. Enabling a syscall tracing event records its fields in a per-CPU tracing buffer.



Tracefs exposes configuration, filters and captured data as files under `/sys/kernel/tracing`.

`current_tracer=nop` disables function tracing, **not enabled events**. `tracing_on` controls recording into the tracing buffer.

Prepare an isolated event capture

Start QEMU: use `make dev-run` with the host-terminal frontend, or run the `AOS: Run QEMU (1 vCPU)` task in the browser. Both open the guest console used below.

Guest: prepare the output sink and a named tracing instance:

```
[ -e /dev/null ] || mknod /dev/null c 1 3
[ -d /sys/kernel/tracing/events ] || mount -t tracefs nodev /sys/kernel/tracing
mkdir -p /sys/kernel/tracing/instances/lab2
T=/sys/kernel/tracing/instances/lab2
export T
echo 0 > "$T/tracing_on"
echo nop > "$T/current_tracer"
echo 0 > "$T/events/enable"
echo > "$T/trace"
```

This starts with recording stopped, no enabled events and an empty buffer. Keep the same guest shell for the following commands.

Select the events

Record entry and exit for opening and transferring file data:

```
echo 1 > "$T/events/syscalls/sys_enter_openat/enable"
echo 1 > "$T/events/syscalls/sys_exit_openat/enable"
echo 1 > "$T/events/syscalls/sys_enter_read/enable"
echo 1 > "$T/events/syscalls/sys_exit_read/enable"
echo 1 > "$T/events/syscalls/sys_enter_sendfile64/enable"
echo 1 > "$T/events/syscalls/sys_exit_sendfile64/enable"
cat "$T/events/syscalls/sys_enter_sendfile64/format"
```

The event format names the fields: `out_fd`, `in_fd`, `offset`, `count`.

We include both `read` and `sendfile64` rather than assuming how this BusyBox build implements `cat`.

Capture only the workload process

```
wc -c /init
sh -c '
  echo $$ > "$T/set_event_pid"
  echo 1 > "$T/tracing_on"
  exec cat /init
' > /dev/null
echo 0 > "$T/tracing_on"
cat "$T/trace"
cat "$T/per_cpu/cpu0/stats"
```

The child shell selects its own PID before enabling recording. `exec` replaces it with `cat` without changing that PID.

The output redirection is established before the child enables tracing. Stopping recording before reading `trace` separates acquisition from inspection.

Read the syscall trace

Observed sequence — selected fields, hexadecimal values:

```
sys_openat(...)
sys_openat -> 0x3
sys_sendfile64(out_fd: 1, in_fd: 3, ...)
sys_sendfile64 -> 0x248
sys_sendfile64(out_fd: 1, in_fd: 3, ...)
sys_sendfile64 -> 0x0
```

All six records belong to `cat`, PID 81 in this run. No `read` events were recorded.

Full syscall capture

- `openat` returns file descriptor **3**.
- `sendfile` copies from descriptor **3** to **1** (stdout, redirected to `/dev/null`).
- `0x248` = **584 bytes**, the file size.
- The next call returns **0**: no more bytes to transfer.

The trace contains the filename **pointer**, not the pathname string. We know the command; the pointer alone does not identify `/init`.

Buffer statistics: **6/6 entries, 0 overruns, 0 dropped events**.

Finish the event capture

```
echo 0 > "$T/events/enable"
echo > "$T/set_event_pid"
```

Recording was already stopped before inspecting the trace. Disable the selected events and remove the PID filter before the next experiment.

The result changes our initial expectation: **this BusyBox `cat` transfers the file with `sendfile`, not a userspace read/write loop**.

Complete guest recipe

Function graph tracing

Select the kernel path

We observed two `sendfile` calls. Now inspect the kernel function **`do_sendfile`** while executing the same workload.

```
T=/sys/kernel/tracing
export T
echo 0 > "$T/tracing_on"
echo 0 > "$T/events/enable"
echo > "$T/set_event_pid"
echo nop > "$T/current_tracer"
echo > "$T/set_ftrace_filter"
echo do_sendfile > "$T/set_graph_function"
echo 3 > "$T/max_graph_depth"
echo function_graph > "$T/current_tracer"
echo > "$T/trace"
```

Use the **top-level tracefs instance**: the named instance lacks these graph controls on our kernel. The root-function filter limits the traced call tree; depth 3 limits its expansion.

Capture and reset the function graph

Run and inspect

```
sh -c '
  echo $$ > "$T/set_ftrace_pid"
  echo 1 > "$T/tracing_on"
  exec cat /init
' > /dev/null
echo 0 > "$T/tracing_on"
cat "$T/trace"
cat "$T/per_cpu/cpu0/stats"
```

`set_ftrace_pid` filters function tracing; it is distinct from the event filter `set_event_pid`.

Then reset

```
echo nop > "$T/current_tracer"
echo > "$T/set_ftrace_pid"
echo > "$T/set_graph_function"
echo 0 > "$T/max_graph_depth"
```

This removes the tracer, PID filter, root-function filter and depth limit used by this experiment.

Complete guest recipe

Read the call tree

Selected lines from the first `do_sendfile` call; `...` marks omitted lines:

```
0)               |  do_sendfile() {
0)               |    __fdget() {
0) + 48.168 us   |      __fget_light();
...
0)               |    rw_verify_area() {
0)   8.077 us    |      security_file_permission();
...
0)               |    do_splice_direct() {
0)   4.962 us    |      rw_verify_area();
0) ! 415.313 us  |      splice_direct_to_actor();
0) ! 460.867 us  |    }
0) # 1475.535 us |  }
```

Indentation represents nested calls. The graph exposes descriptor lookup, access checks and the kernel's splice path; it does not print syscall argument values.

Calls, interrupts and durations

- The capture has **two** `do_sendfile` **roots**, consistent with the two syscall invocations observed in the separate event capture.
- A second call still executes kernel code even when it ultimately transfers zero bytes.
- The full graph also contains a timer interrupt nested during execution. The PID filter does not make interrupt activity disappear.
- Durations are from **instrumented QEMU execution**, not native-machine performance measurements. Nested durations must not simply be added together.

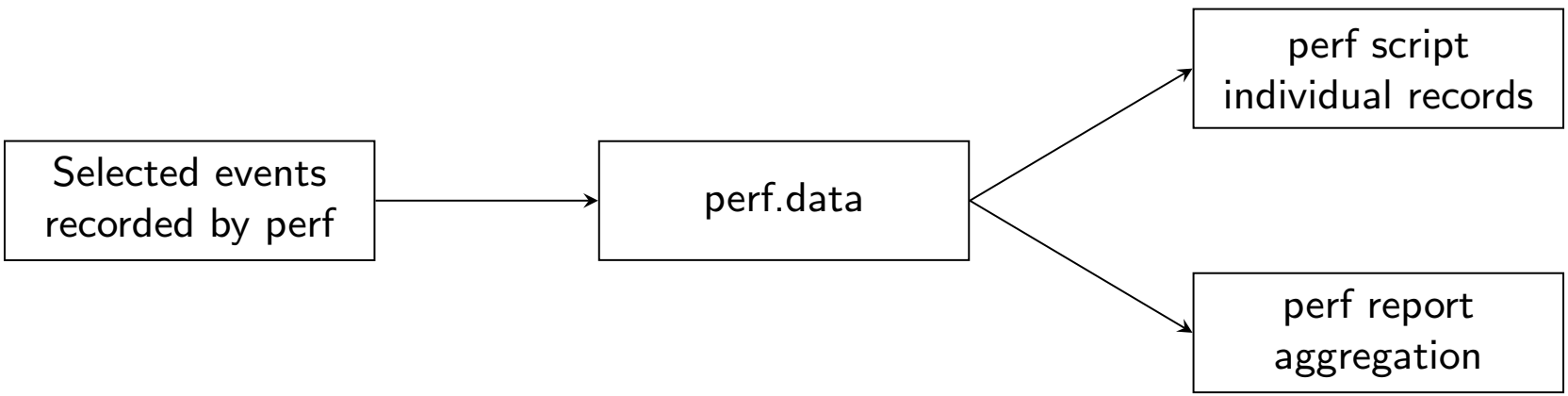
The depth limit hides deeper calls. The captured buffer reported **58 entries, 0 overruns and 0 dropped events**.

Full function graph capture

Recording and analysing events with perf

Record now, inspect afterwards

Tracefs let us read its tracing buffer directly. **Perf records selected events into `perf.data` through the perf event interface**, using its own recording buffers.



The image already provides `perf`; the verified version is `5.16.0-rc1`. These syscall events do not require hardware performance counters or a BPF program.

Record the same workload

Guest, after resetting the previous capture:

```
mkdir -p /tmp
export PERF_PAGER=cat
perf --version
perf record -o /tmp/lab2-perf.data -c 1 \
  -e syscalls:sys_enter_openat -e syscalls:sys_exit_openat \
  -e syscalls:sys_enter_sendfile64 -e syscalls:sys_exit_sendfile64 \
  -- cat /init > /dev/null
```

- `-e` selects the events; `-c 1` requests a sample for every occurrence.
- Without `-a`, this command records the launched workload, not all tasks system-wide.
- We record **events**, not periodic CPU samples. No `cycles` event is requested.

Observed: `Captured and wrote 0.002 MB /tmp/lab2-perf.data (6 samples)`.

Inspect individual records

```
perf script -i /tmp/lab2-perf.data > /tmp/lab2-perf-script.txt
cat /tmp/lab2-perf-script.txt
```

Selected event names and results from the capture, all belonging to `cat` PID 92:

```
syscalls:sys_enter_openat
syscalls:sys_exit_openat       0x3
syscalls:sys_enter_sendfile64
syscalls:sys_exit_sendfile64   0x248
syscalls:sys_enter_sendfile64
syscalls:sys_exit_sendfile64   0x0
```

The sequence agrees with the earlier tracefs capture. These are separate runs: their PIDs, addresses and timestamps differ.

Redirecting output to a file avoids interactive paging and terminal-width truncation during capture.

Interpret the aggregation

```
perf report -i /tmp/lab2-perf.data --stdio --sort comm \
  --show-nr-samples --percent-limit 0 > /tmp/lab2-perf-report.txt
cat /tmp/lab2-perf-report.txt
```

Selected lines from the report:

```
# Samples: 2 of event
# 'syscalls:sys_exit_sendfile64'
# Event count (approx.): 2
# Overhead  Samples  Command
  100.00%        2  cat
```

Event-name line wrapped for readability.

The four event groups contain **1, 1, 2 and 2 samples** respectively. Total lost samples: **0**.

`100%` means that `cat` accounts for all samples **of this selected event**. It is not a CPU-time percentage.

An entry and its exit are separate records: six samples here represent three syscall invocations, not six invocations.

Reproduce and compare

All commands were exercised in the same amd64 QEMU environment. Full guest recipes and captured output are stored beside this handout:

- Syscall commands and trace.
- Function graph commands and trace.
- Perf commands and output.

The scripts contain **guest commands**. They are not automatically installed in the initramfs; enter the commands in the QEMU shell, or copy the scripts into a prepared guest.

The three views answer different questions: **requests and results**, **nested kernel calls**, **recorded events and their aggregation**.

Linux event tracing · Function tracing · Perf examples